CS505PC: DEVOPS LAB

EXPERIMENT – 1

Write Code for a Simple User Registration Form for an Event

1. AIM

To develop a simple event user-registration form using HTML and CSS and validate the form using basic browser controls.

2. PROCEDURE

1. Create a folder named event-registration.
2. Inside the folder, create a file named index.html.
3. Add the HTML code given in the Program section.
4. Save the file and open index.html in a web browser.
5. Enter a name, email address, phone number, and event choice.
6. Click Register and verify that the browser validates required fields and the email format.
7. Record the displayed registration confirmation.

3. PROGRAM / COMMANDS

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Event Registration</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 480px; margin: 40px auto; }
    label { display: block; margin-top: 12px; }
    input, select, button { box-sizing: border-box; width: 100%; padding: 9px; margin-top:
5px; }
    button { margin-top: 18px; cursor: pointer; }
  </style>
</head>
<body>
  <h2>Event Registration Form</h2>
  <form onsubmit="event.preventDefault();
document.getElementById('message').textContent='Registration submitted successfully!';">
    <label>Full Name
      <input type="text" name="name" required>
    </label>
    <label>Email
      <input type="email" name="email" required>
    </label>
    <label>Phone Number
      <input type="tel" name="phone" pattern="[0-9]{10}" title="Enter a 10-digit phone
number" required>
    </label>
    <label>Event
      <select name="event" required>
```

```
        <option value="">Select an event</option>
        <option>Workshop</option>
        <option>Seminar</option>
        <option>Hackathon</option>
      </select>
    </label>
    <button type="submit">Register</button>
  </form>
  <p id="message" role="status"></p>
</body>
</html>
```

4. EXPECTED OUTPUT

The browser displays an Event Registration Form with Full Name, Email, Phone Number, Event, and Register fields. On valid submission, it displays: Registration submitted successfully!

5. RESULT

A simple event registration form was created and tested successfully in a web browser.

CS505PC: DEVOPS LAB

EXPERIMENT – 2

Explore Git and GitHub Commands

1. AIM

To explore basic Git commands for initializing a repository, tracking files, creating commits, and connecting a local repository to GitHub.

2. PROCEDURE

1. Install Git and verify the installation using git --version.
2. Create a folder named git-practice and open a terminal in that folder.
3. Configure the Git username and email.
4. Initialize a local Git repository.
5. Create a README.md file and check the repository status.
6. Stage the file and commit it.
7. Create an empty repository on GitHub, then add its URL as the remote.
8. Push the local commit to GitHub and verify the file on the GitHub website.

3. PROGRAM / COMMANDS

```
# Verify Git
git --version

# Configure identity (use your own details)
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# Create and enter a project folder
mkdir git-practice
cd git-practice

# Initialize repository
git init

# Create a sample file (Linux/macOS/Git Bash)
echo "# Git Practice" > README.md

# Check, stage, and commit
git status
git add README.md
git commit -m "Add README"

# After creating an empty repository on GitHub, replace the URL below
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/git-practice.git
git push -u origin main
```

4. EXPECTED OUTPUT

git --version displays the installed Git version. git status shows README.md as untracked before staging and a clean working tree after committing. git log shows the commit. After a successful push, README.md is visible in the GitHub repository.

5. RESULT

Basic Git commands were executed, a local commit was created, and the repository was pushed to GitHub.

CS505PC: DEVOPS LAB

EXPERIMENT – 3

Practice Source Code Management on GitHub Using Experiment 1

1. AIM

To practice source code management by uploading the event registration form created in Experiment 1 to GitHub and tracking changes using Git.

2. PROCEDURE

1. Open the event-registration folder created in Experiment 1.
2. Initialize Git in the folder.
3. Configure Git username and email if not already configured.
4. Stage and commit index.html.
5. Create an empty GitHub repository named event-registration.
6. Connect the local repository to the GitHub repository and push the main branch.
7. Edit index.html, for example by changing the page heading.
8. Stage and commit the modification, then push it to GitHub.
9. Refresh the GitHub repository page and verify the latest version and commit history.

3. PROGRAM / COMMANDS

```
# Run these commands inside the event-registration folder
git init
git config user.name "Your Name"
git config user.email "you@example.com"

git status
git add index.html
git commit -m "Add event registration form"

# Replace with your own GitHub repository URL
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/event-registration.git
git push -u origin main

# After editing index.html, record the change
git add index.html
git commit -m "Update registration form"
git push
```

4. EXPECTED OUTPUT

The event-registration repository appears on GitHub with index.html. After modifying and pushing the file, GitHub shows the updated source code and two commits in the history.

5. RESULT

The event registration application's source code was managed using Git and GitHub, including committing and pushing a change.

CS505PC: DEVOPS LAB

EXPERIMENT – 4

Jenkins Installation and Setup; Explore the Environment

1. AIM

To install and start Jenkins, open its web interface, complete the initial setup, and explore the Jenkins environment.

2. PROCEDURE

1. Install a supported Java Development Kit (JDK) on a suitable machine and verify Java with java -version.
2. Download and install Jenkins using the official instructions for the operating system, or run the official Jenkins LTS container if Docker is already installed.
3. Start the Jenkins service or container.
4. Open http://localhost:8080 in a browser, or use the host address and port if Jenkins runs on another machine.
5. Retrieve the initial administrator password from the path shown by Jenkins or from the container logs.
6. Enter the password, install suggested plugins, and create an administrator account.
7. Open the Jenkins Dashboard and explore New Item, Build History, Manage Jenkins, and plugins.

3. PROGRAM / COMMANDS

```
# Example: run Jenkins LTS in Docker (requires Docker installed)
docker volume create jenkins_home
docker run -d --name jenkins-lab -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts

# Display the initial setup password from container logs
docker exec jenkins-lab cat /var/jenkins_home/secrets/initialAdminPassword

# Open in a browser:
http://localhost:8080
```

4. EXPECTED OUTPUT

The Jenkins Unlock page appears at http://localhost:8080. After entering the initial password, installing plugins, and creating an administrator, the Jenkins Dashboard is displayed.

5. RESULT

Jenkins was installed and started successfully, initial setup was completed, and the Jenkins dashboard was explored.

CS505PC: DEVOPS LAB

EXPERIMENT – 5

Demonstrate Continuous Integration and Development Using Jenkins

1. AIM

To demonstrate a basic Continuous Integration workflow in Jenkins by creating a job that checks out source code and runs a simple build step.

2. PROCEDURE

1. Ensure Jenkins is running and the initial setup is complete.
2. Create a small project repository on GitHub containing a README.md file.
3. In Jenkins, select New Item, enter a job name, and choose Pipeline.
4. In the Pipeline section, choose Pipeline script and enter the Jenkinsfile script provided below.
5. Save the job.
6. Click Build Now and open the build number in Console Output.
7. Verify that the pipeline checks out source code and prints a successful build message.
8. Modify the message or source repository and run the job again to observe a new build.

3. PROGRAM / COMMANDS

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                echo 'Checking out source code'
                // Replace with your repository URL
                git branch: 'main',
                    url: 'https://github.com/YOUR-USERNAME/YOUR-REPOSITORY.git'
            }
        }
        stage('Build') {
            steps {
                echo 'Build stage completed'
            }
        }
        stage('Test') {
            steps {
                echo 'Test stage completed'
            }
        }
    }
    post {
        success {
            echo 'CI pipeline completed successfully'
        }
        failure {
            echo 'CI pipeline failed; check the console output'
        }
```

```
        }
}
```

4. EXPECTED OUTPUT

The Jenkins Console Output shows Checkout, Build, and Test stages completing. The final message is: CI pipeline completed successfully. The build status is SUCCESS if the repository URL and job configuration are valid.

5. RESULT

A basic Jenkins CI pipeline was configured and executed successfully to check out source code and run build/test stages.

CS505PC: DEVOPS LAB

EXPERIMENT – 6

Explore Docker Commands for Container Management

1. AIM

To explore basic Docker commands for downloading images, creating and running containers, inspecting them, viewing logs, and removing resources.

2. PROCEDURE

1. Install Docker Desktop or Docker Engine and start the Docker service.
2. Verify the installation using docker --version and docker info.
3. Download the Nginx image from Docker Hub.
4. List locally available images.
5. Run an Nginx container and map host port 8081 to container port 80.
6. List running containers and inspect container logs.
7. Open http://localhost:8081 in a browser to verify the web server.
8. Stop and remove the container, then optionally remove the image.

3. PROGRAM / COMMANDS

```
# Verify Docker
docker --version
docker info

# Download and list an image
docker pull nginx
docker images

# Run Nginx; host port 8081 maps to container port 80
docker run -d --name nginx-lab -p 8081:80 nginx

# Inspect running containers and logs
docker ps
docker logs nginx-lab

# Open http://localhost:8081 in a browser

# Stop and remove the container
docker stop nginx-lab
docker rm nginx-lab

# Optional: remove the downloaded image
docker rmi nginx
```

4. EXPECTED OUTPUT

docker images lists the Nginx image. docker ps lists nginx-lab as running. The browser at http://localhost:8081 displays the Nginx welcome page. After cleanup, docker ps -a no longer lists nginx-lab.

5. RESULT

Basic Docker image and container-management commands were executed and verified successfully.

CS505PC: DEVOPS LAB

EXPERIMENT – 7

Develop a Simple Containerized Application Using Docker

1. AIM

To develop a simple web application, package it into a Docker image, and run it inside a Docker container.

2. PROCEDURE

1. Install and start Docker.
2. Create a folder named docker-web-app.
3. Inside it, create index.html containing a simple welcome message.
4. Create a Dockerfile using the Nginx base image and copy index.html to the web root.
5. Open a terminal in the folder and build the image.
6. Run a container with host port 8082 mapped to container port 80.
7. Open http://localhost:8082 and verify the application.
8. Stop and remove the container after testing.

3. PROGRAM / COMMANDS

```
# File: index.html
<!DOCTYPE html>
<html>
<head><title>Docker Lab</title></head>
<body>
  <h1>Hello from a Docker Container!</h1>
  <p>This web application is served by Nginx.</p>
</body>
</html>

# File: Dockerfile
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html

# Run these commands in the folder containing both files
docker build -t docker-web-app:1.0 .
docker run -d --name docker-web-lab -p 8082:80 docker-web-app:1.0

# Open http://localhost:8082
# Cleanup
docker stop docker-web-lab
docker rm docker-web-lab
```

4. EXPECTED OUTPUT

The image docker-web-app:1.0 is built successfully. The browser at http://localhost:8082 displays “Hello from a Docker Container!” and the supporting text.

5. RESULT

A simple web application was packaged into a Docker image and executed successfully inside a container.

CS505PC: DEVOPS LAB

EXPERIMENT – 8

Integrate Kubernetes and Docker

1. AIM

To prepare a local Kubernetes environment and deploy a Docker-containerized application to a Kubernetes cluster.

2. PROCEDURE

1. Install Docker Desktop with Kubernetes enabled, or install a local cluster tool such as Minikube and kubectl.
2. Start the local Kubernetes cluster and verify it using kubectl cluster-info.
3. Confirm that the cluster nodes are Ready using kubectl get nodes.
4. Use the Docker image created in Experiment 7. For a local cluster, ensure the image is available to the cluster; with Minikube, build the image in Minikube's Docker environment or load it into the cluster.
5. Create a Kubernetes Deployment for the image.
6. Expose the Deployment as a Service.
7. Verify the Deployment, Pods, and Service using kubectl commands.
8. Access the application through the service URL or mapped NodePort, depending on the cluster setup.

3. PROGRAM / COMMANDS

```
# Verify cluster
kubectl cluster-info
kubectl get nodes

# If using Minikube, make the local image available:
# minikube image load docker-web-app:1.0

# File: deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: docker-web-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: docker-web
  template:
    metadata:
      labels:
        app: docker-web
    spec:
      containers:
      - name: web
        image: docker-web-app:1.0
        imagePullPolicy: IfNotPresent
```

```
        ports:
        - containerPort: 80

# File: service.yaml
apiVersion: v1
kind: Service
metadata:
  name: docker-web-service
spec:
  type: NodePort
  selector:
    app: docker-web
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080

# Apply and verify
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get deployments
kubectl get pods
kubectl get services
```

4. EXPECTED OUTPUT

kubectl get nodes shows a Ready node. kubectl get deployments shows docker-web-deployment with one available replica. kubectl get pods shows the application Pod as Running, and kubectl get services lists docker-web-service.

5. RESULT

A Docker-containerized application was deployed to a local Kubernetes cluster and its Deployment, Pod, and Service were verified.

CS505PC: DEVOPS LAB

EXPERIMENT – 9

Automate Running the Containerized Application Using Kubernetes

1. AIM

To automate the deployment and running of the containerized application developed in Experiment 7 using Kubernetes Deployment and Service resources.

2. PROCEDURE

1. Start the Kubernetes cluster and verify that its node is Ready.
2. Ensure the docker-web-app:1.0 image from Experiment 7 is available to the cluster.
3. Create a Deployment YAML file that specifies the container image and desired replica count.
4. Create a Service YAML file to expose the application.
5. Apply the configuration files using kubectl apply.
6. Check rollout status, Pods, and Service.
7. Scale the Deployment to two replicas and verify the desired number of Pods.
8. Delete the Kubernetes resources after the experiment.

3. PROGRAM / COMMANDS

```
# Make sure the local image is available to the cluster first.
# For Minikube: minikube image load docker-web-app:1.0

# File: app.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: event-web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: event-web
  template:
    metadata:
      labels:
        app: event-web
    spec:
      containers:
      - name: event-web
        image: docker-web-app:1.0
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: event-web-service
```

```
spec:
  type: NodePort
  selector:
    app: event-web
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30081

# Deploy and verify
kubectl apply -f app.yaml
kubectl rollout status deployment/event-web
kubectl get deployments
kubectl get pods
kubectl get services

# Scale the application
kubectl scale deployment/event-web --replicas=2
kubectl get pods

# Cleanup
kubectl delete -f app.yaml
```

4. EXPECTED OUTPUT

kubectl rollout status reports that the deployment has successfully rolled out. kubectl get deployments shows 2 desired and 2 available replicas; kubectl get pods lists two Running Pods after startup.

5. RESULT

The containerized application was deployed and managed automatically using Kubernetes resources, and scaling was verified.

CS505PC: DEVOPS LAB

EXPERIMENT – 10

Install and Explore Selenium for Automated Testing

1. AIM

To install Selenium WebDriver and explore browser automation by opening a web page and verifying its title using Python.

2. PROCEDURE

1. Install Python 3 and a supported web browser such as Chrome or Firefox.
2. Create a project folder and open a terminal in it.
3. Create and activate a Python virtual environment (recommended).
4. Install Selenium using pip.
5. Create a Python file named test_selenium.py containing the program below.
6. Run the program. Selenium Manager can obtain the appropriate driver in supported Selenium versions, subject to network access and browser compatibility.
7. Observe the browser opening the page and the page title printed in the terminal.

3. PROGRAM / COMMANDS

```
# Install Selenium first:
# python -m pip install -U selenium

# File: test_selenium.py
from selenium import webdriver

driver = webdriver.Chrome()
try:
    driver.get("https://www.selenium.dev/")
    print("Page title:", driver.title)
    assert "Selenium" in driver.title
    print("Title verification passed")
finally:
    driver.quit()

# Run:
# python test_selenium.py
```

4. EXPECTED OUTPUT

A browser window opens the Selenium website. The terminal prints the page title (which may vary if the website title changes) and then prints: Title verification passed.

5. RESULT

Selenium was installed and used to automate a browser, open a web page, and verify its title.

CS505PC: DEVOPS LAB

EXPERIMENT – 11

Write a Simple Program in JavaScript and Perform Testing Using Selenium

1. AIM

To develop a simple JavaScript program for validating a user name and email address, and test its behaviour automatically using Selenium WebDriver.

2. PROCEDURE

1. Install Python 3 and a supported browser such as Chrome.
2. Create a folder named selenium-javascript-lab.
3. Inside the folder, create an HTML file named index.html using the program below.
4. The page contains a JavaScript form that checks whether the name and email fields are entered.
5. Install Selenium in the Python environment using python -m pip install -U selenium.
6. Create a Python file named test_form.py using the Selenium test program below.
7. Run the HTML page from a local web server. From the project folder, run python -m http.server 8000.
8. Open a second terminal in the same folder and run python test_form.py.
9. Verify that Selenium tests an empty submission and a valid submission, and prints the test results.

3. PROGRAM

A. HTML and JavaScript (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>JavaScript Form Test</title>
</head>
<body>
  <h1>Registration Form</h1>
  <form id="registrationForm">
    <label>Name:
      <input id="name" type="text">
    </label>
    <br><br>
    <label>Email:
      <input id="email" type="text">
    </label>
    <br><br>
    <button type="submit">Submit</button>
  </form>
  <p id="message" role="status"></p>

  <script>
    document.getElementById("registrationForm").addEventListener("submit", function(event)
{
      event.preventDefault();
```

```
      const name = document.getElementById("name").value.trim();
      const email = document.getElementById("email").value.trim();
      const message = document.getElementById("message");

      if (name === "" || email === "") {
        message.textContent = "Please enter name and email.";
      } else if (!email.includes("@") || !email.includes(".")) {
        message.textContent = "Please enter a valid email.";
      } else {
        message.textContent = "Form submitted successfully!";
      }
    });
  </script>
</body>
</html>
```

B. Selenium Test Program (test_form.py)

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

try:
    driver.get("http://127.0.0.1:8000/index.html")

    # Test 1: Submit with empty fields
    driver.find_element(By.ID, "name").clear()
    driver.find_element(By.ID, "email").clear()
    driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
    message = driver.find_element(By.ID, "message").text
    assert message == "Please enter name and email."
    print("Test 1 passed: Empty fields are rejected")

    # Test 2: Submit valid details
    driver.find_element(By.ID, "name").send_keys("Ravi")
    driver.find_element(By.ID, "email").send_keys("ravi@example.com")
    driver.find_element(By.CSS_SELECTOR, "button[type='submit']").click()
    message = driver.find_element(By.ID, "message").text
    assert message == "Form submitted successfully!"
    print("Test 2 passed: Valid details are accepted")
finally:
    driver.quit()
```

4. EXPECTED OUTPUT

The browser opens the registration form. The Python terminal displays:

```
Test 1 passed: Empty fields are rejected
Test 2 passed: Valid details are accepted
```

5. RESULT

A simple JavaScript form was developed and tested using Selenium WebDriver. The automated tests verified that empty fields are rejected and valid details are accepted.

CS505PC: DEVOPS LAB

EXPERIMENT – 12

Develop Test Cases for the Containerized Application Using Selenium

1. AIM

To develop and execute Selenium test cases for the containerized web application created in Experiment 7, and verify that the application loads and displays the expected content in a browser.

2. PROCEDURE

1. Ensure Docker is installed and running.
2. Create the application files from Experiment 7: index.html and Dockerfile. The HTML page should display the heading 'Hello from a Docker Container!' and the text 'This web application is served by Nginx.'
3. Build the Docker image and run the container, mapping host port 8082 to container port 80.
4. Verify that the application opens at http://localhost:8082 in a browser.
5. Install Python 3 and Selenium using python -m pip install -U selenium.
6. Create a Python file named test_container_app.py using the Selenium test program below.
7. Keep the Docker container running and execute python test_container_app.py.
8. Check the terminal output to verify the page title, heading, supporting text, and successful HTTP page load.
9. Stop and remove the container after the tests are complete.

3. PROGRAM / COMMANDS

A. Build and Run the Container (if not already running)

```
docker build -t docker-web-app:1.0 .
docker run -d --name docker-web-lab -p 8082:80 docker-web-app:1.0

# Confirm the container is running
docker ps
```

B. Selenium Test Program (test_container_app.py)

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

try:
    driver.get("http://localhost:8082")

    # Test 1: Verify page title
    assert "Docker Lab" in driver.title
    print("Test 1 passed: Page title is correct")

    # Test 2: Verify main heading
    heading = driver.find_element(By.TAG_NAME, "h1").text
```

```
    assert heading == "Hello from a Docker Container!"
    print("Test 2 passed: Main heading is displayed")

    # Test 3: Verify supporting text
    paragraph = driver.find_element(By.TAG_NAME, "p").text
    assert paragraph == "This web application is served by Nginx."
    print("Test 3 passed: Supporting text is displayed")

    # Test 4: Verify page URL
    assert driver.current_url.startswith("http://localhost:8082")
    print("Test 4 passed: Application URL is correct")
finally:
    driver.quit()

# After testing, clean up the container from a terminal:
# docker stop docker-web-lab
# docker rm docker-web-lab
```

4. EXPECTED OUTPUT

The browser opens the containerized application. The Python terminal displays:

```
Test 1 passed: Page title is correct
Test 2 passed: Main heading is displayed
Test 3 passed: Supporting text is displayed
Test 4 passed: Application URL is correct
```

5. RESULT

Selenium test cases were developed and executed for the Docker-based web application. The page title, main heading, supporting text, and application URL were verified successfully.

DEVOPS LAB — PROGRAM OUTPUTS

Clarification: exact runtime output is only deterministic for fixed program-generated text. Git hashes, Docker container IDs, Kubernetes pod names, software versions, logs, and environment details are created at runtime, so they cannot honestly be specified as exact in advance. This section gives the exact fixed text produced by the supplied programs and identifies environment-dependent output.

EXPERIMENT 1 — Event Registration Form

Exact deterministic output

```
Browser page:
Event Registration Form

Full Name: [input field]
Email: [input field]
Phone Number: [input field]
Event: [Select an event ▼]
Register

After valid input and clicking Register:
Registration submitted successfully!
```

Important

```
The page layout depends on the browser and window size. A specific screenshot cannot be identical on every device.
```

EXPERIMENT 2 — Git and GitHub Commands

Exact output limits

```
git --version
Output depends on the installed Git version, for example: git version 2.x.x

git status
Output depends on whether the repository is new, staged, committed, and on the current branch.

git commit
The commit hash and file-change counts are generated locally.

git push
The output depends on your GitHub username, repository URL, authentication, and remote state.
```

What to record

```
Run each command on your computer and copy the actual terminal output. These outputs cannot be known exactly without ru
```

EXPERIMENT 3 — GitHub Source Code Management

Exact output limits

```
git init → reports the local repository path and initialization status.
git config → normally prints nothing when successful.
git status → reports the current branch and file state.
git commit → reports a generated commit hash and change counts.
git push → reports the remote branch update or an error if authentication/URL is wrong.
```

What to record

```
Exact commit hashes, paths, branch state, and push messages are generated by your machine and GitHub repository; they c
```

EXPERIMENT 4 — Jenkins Installation and Setup

Exact output limits

```
docker volume create jenkins_home
→ prints the volume name if creation succeeds.

docker run ...
→ prints a generated container ID.

docker exec jenkins-lab cat /var/jenkins_home/secrets/initialAdminPassword
→ prints a generated initial administrator password.

Browser → Jenkins Unlock page, followed by setup screens and dashboard.
```

What to record

```
The actual container ID, password, Jenkins version, plugin screen, and dashboard are specific to your installation.
```

EXPERIMENT 5 — Jenkins CI Pipeline

Output from the program's echo steps

```
Checking out source code
Build stage completed
Test stage completed
CI pipeline completed successfully
```

Important

```
Jenkins also prints timestamps, stage metadata, checkout logs, and build status. Those details vary by Jenkins version
```

EXPERIMENT 6 — Docker Commands

Exact output limits

```
docker --version → installed Docker version.
docker info → host-specific Docker configuration and status.
docker pull nginx → pull/download status depends on cache and network.
docker images → local image IDs, tags, sizes, and dates.
docker run ... → generated container ID.
docker ps → current running container details.
docker logs nginx-lab → actual Nginx logs, which depend on requests and timing.
docker stop / docker rm / docker rmi → status messages depend on resource state.
```

Browser output

```
If the container starts correctly, http://localhost:8081 displays the Nginx welcome page. Exact formatting may vary wit
```

EXPERIMENT 7 — Containerized Web Application

Browser output from index.html

```
Hello from a Docker Container!
This web application is served by Nginx.
```

Exact command output limits

```
docker build prints build-step logs, and docker run prints a generated container ID. Both vary by Docker version, cache
```

EXPERIMENT 8 — Kubernetes and Docker

Exact output limits

kubectl cluster-info → cluster-specific URLs.
kubectl get nodes → actual node names, status, age, and version.
kubectl apply → created/configured/unchanged messages based on current resources.
kubectl get deployments/pods/services → actual names, readiness, IPs, ages, and ports.

Important

The deployment can only run successfully if the Kubernetes cluster can access the image docker-web-app:1.0. Pod names a

EXPERIMENT 9 — Kubernetes Automation

Expected state after successful execution

```
Deployment event-web: 2 desired replicas and 2 available replicas.
Two application Pods should reach Running.
Service event-web-service should be created with NodePort 30081.
Cleanup deletes the Deployment and Service.
```

Exact output limits

```
Pod names, timing, resource messages, and rollout status depend on the cluster's actual state.
```

EXPERIMENT 10 — Selenium Browser Test

Program's printed output when the page title contains Selenium

```
Page title: Selenium
Title verification passed
```

Exact output limits

```
The title is read live from https://www.selenium.dev/ and may change. Browser startup errors or network problems will c
```

EXPERIMENT 11 — JavaScript Form with Selenium

Program's printed output when both assertions pass

```
Test 1 passed: Empty fields are rejected
Test 2 passed: Valid details are accepted
```

Browser messages

```
Empty submission:
Please enter name and email.

Valid submission:
Form submitted successfully!
```

EXPERIMENT 12 — Selenium Tests for Containerized Application

Program's printed output when all assertions pass

```
Test 1 passed: Page title is correct
Test 2 passed: Main heading is displayed
Test 3 passed: Supporting text is displayed
Test 4 passed: Application URL is correct
```

Browser content

```
Title: Docker Lab
Heading: Hello from a Docker Container!
Text: This web application is served by Nginx.
```

Exact output limits

```
These four success lines appear only when the container is running on localhost:8082 and all assertions pass.
```